

中国海洋大学计算机科学与技术学院

实验一 DataLab 实验报告

姓名：庄环嘉 年级：2022 级 专业：计算机科学与技术

科目：计算机系统基础 题目：Data Lab1

实验时间：2024 年 10 月 25 日 实验教师：蒋永国

一、实验目的：

1. 熟悉和掌握计算机中整数和浮点数的二进制编码表示

二、实验平台：

1. Educoder 在线实践教学平台

三、实验任务及要求：

实验要求

- 整数
 - 只能使用 0-255 的整型常数
 - 只能使用函数参数与函数内声明的局部变量
 - 只能使用如下单目操作符：! ~
 - 只能使用如下双目操作符：& ^ | + << >>
 - 最多只能使用有限个运算符（等于号、括号不计），可以认为使用的运算符个数越少得分越高
 - 一些函数可能对操作符有更多的限制（在题目前以操作符限制给出）
 - 禁止使用任何控制结构如 if do while for switch 等
 - 禁止定义或使用任何宏
 - 禁止定义任何函数
 - 禁止调用任何函数（除了可以使用 printf 输出中间变量，但提交时必须去掉）
 - 禁止使用任何形式的类型转换
 - 禁止使用 int 以外的任何类型（包括结构体、数组、联合体）
- 浮点数
 - 只能使用函数参数与函数内声明的局部变量
 - 最多只能使用有限个运算符（等于号、括号不计），可以认为使用的运算符个数越少得分越高
 - 禁止定义或使用任何宏
 - 禁止定义任何函数
 - 禁止调用任何函数（除了可以使用 printf 输出中间变量，但提交时必须去掉）
 - 禁止使用任何形式的类型转换

1、bitAnd:

补充函数 bitAnd(), 只用~、|实现 $x \& y$, 将结果 return 返回。

1) 操作符限制: ~、|

2) 操作符使用数量限制: 8

2、getByte:

补充函数 getByte(), 取出 x 中的 n 号字节, 将结果 return 返回。

1) 编号从低位到高位从 0 开始

2) 操作符使用数量限制: 6

3、logicalShift

补充函数 logicalShift(), 将 x 逻辑右移 n 位($0 \leq n \leq 31$), 将结果 return 返回。

1) 操作符使用数量限制: 20

4、bitCount:

补充函数 bitCount(), 统计 x 的二进制表示中 1 的数量, 将结果 return 返回。

1) 操作符使用数量限制: 40

5、bang

补充函数 bang(), 不使用!实现!操作符, 将结果 return 返回。

1) 操作符限制: ~ & ^ | + << >>

2) 操作符使用数量限制: 12

6、tmin

补充函数 tmin(), 返回补码表示的整型的最小值, 将结果 return 返回。

操作符使用数量限制: 4

7、fitsBits

补充函数 fitsBits(), 如果 x 可以只用 n 位补码表示则返回 1, 否则返回 0($1 \leq n \leq 32$)。

操作符使用数量限制: 15

8、divpwr2

本关任务: 补充函数 divpwr2(), 计算 $\lfloor x/2^n \rfloor$ ($0 \leq n \leq 30$) 结果向零取整。

操作符使用数量限制: 15

9、negate

补充函数 negate(), 计算 $-x$ 。

操作符使用数量限制: 5

10、isPositive

补充函数 isPositive(), 如果 x 大于 0 返回 1, 否则返回 0。

操作符使用数量限制: 8

11、isLessOrEqual

本关任务: 补充函数 isLessOrEqual(), 如果 x 小于等于 y 则返回 1, 否则返回 0。

操作符使用数量限制: 24

12、ilog2

补充函数 ilog2(), 返回 x 求以 2 为底的对数的结果 (向下取整)。

操作符使用数量限制: 90

13、float_neg

补充函数 float_neg(), 返回 -uf 的位级表示。

操作符使用数量限制: 10

采用 unsigned int 来存放位级表示

所有的浮点类型都为 float

如果输入为 NaN，返回 NaN

14、float_i2f

补充函数 float_i2f(), 实现由 int 到 float 的类型转换。

操作符使用数量限制：30

15、float_twice

补充函数 float_twice(), 返回 2*f 的位级表示。

操作符使用数量限制：30

四、实验过程及结果：

1、bitAnd:

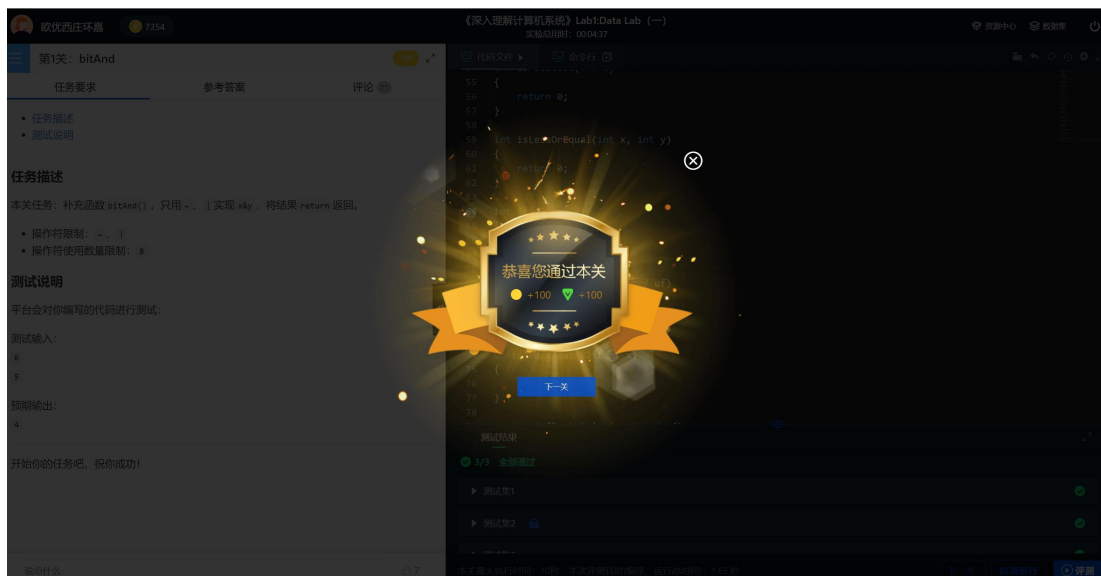
1) 实验代码：

```
int bitAnd(int x, int y)
{
    /******* Begin *****/
    int ret;
    x=~x;
    y=~y;
    ret =x|y;
    ret=~ret;
    return ret;

    /******* End *****/
}
```

2) 实验过程：将题干信息转化为离散数学中的题目，类似于离散数学中求什么情况下用逆事件和 $\cup$ 可以表示 $A \cap B$ ，但是二者有本质的区别，一个是位运算，一个是逻辑运算，只是借用思想（看成每一位与另一个数的对应位相 $\cup$ 或取逆事件）。

3) 实验结果：



2、getByte:

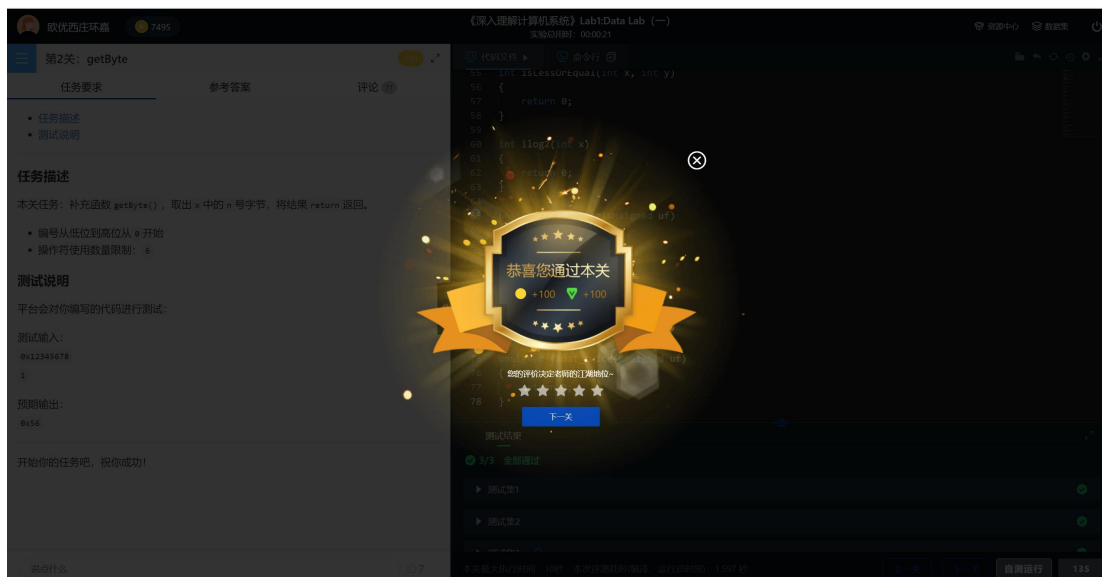
1) 实验代码：

```
int getByte(int x, int n)
{
    /******* Begin *****/
    n=n<<3;
    x=x>>n ;
    return x&0xFF;
    /******* End *****/
}
```

2) 实验过程:

取 n 号字节, 编号从低位到高位从 0 开始, 一个字节占 8 位, 所以需要 $(x \gg (n * 8))$, 而为了排除更高字节的影响, 需要把高位数字都置为 0, 所以 $(x \gg (n * 8)) \& (0xff)$ 。

3) 实验结果:



3、logicalShift:

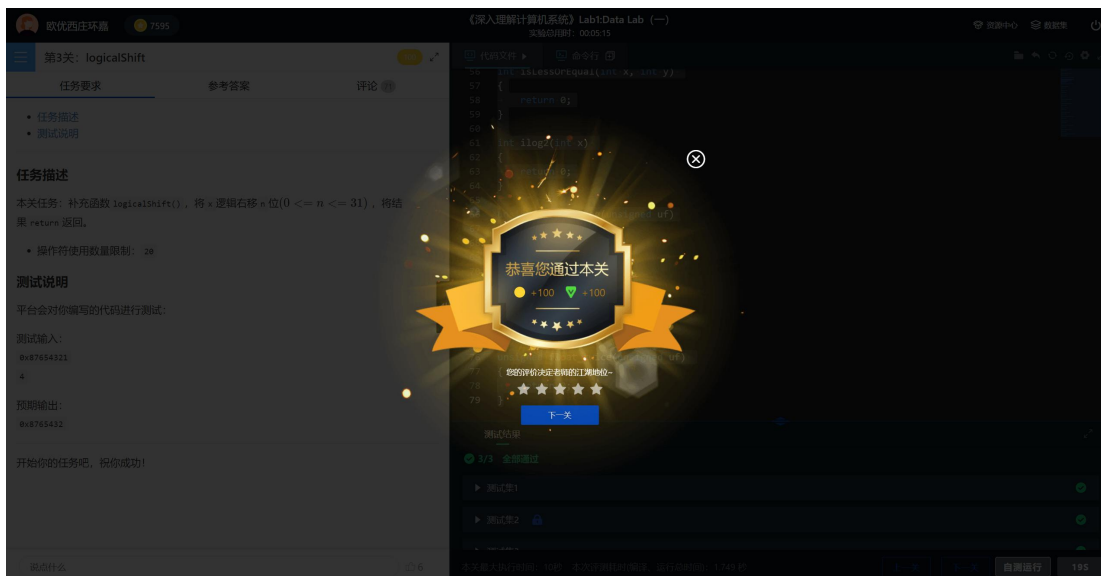
1) 实验代码

```
int logicalShift(int x, int n)
{
    /******* Begin *****/
    unsigned int tmp = x;
    tmp = tmp>>n;
    return tmp;

    /******* End *****/
}
```

2) 实验过程: 依据题意右移四位

3) 实验结果:



4、bitCount:

1) 实验代码:

```
int bitCount(int x)
{
    /******* Begin *****/
    unsigned int d = x;

    d = (d&0x55555555)+((d>>1)&0x55555555);

    d = (d&0x33333333)+((d>>2)&0x33333333);

    d = (d&0x0F0F0F0F)+((d>>4)&0x0F0F0F0F);

    d = (d&0x00FF00FF)+((d>>8)&0x00FF00FF);

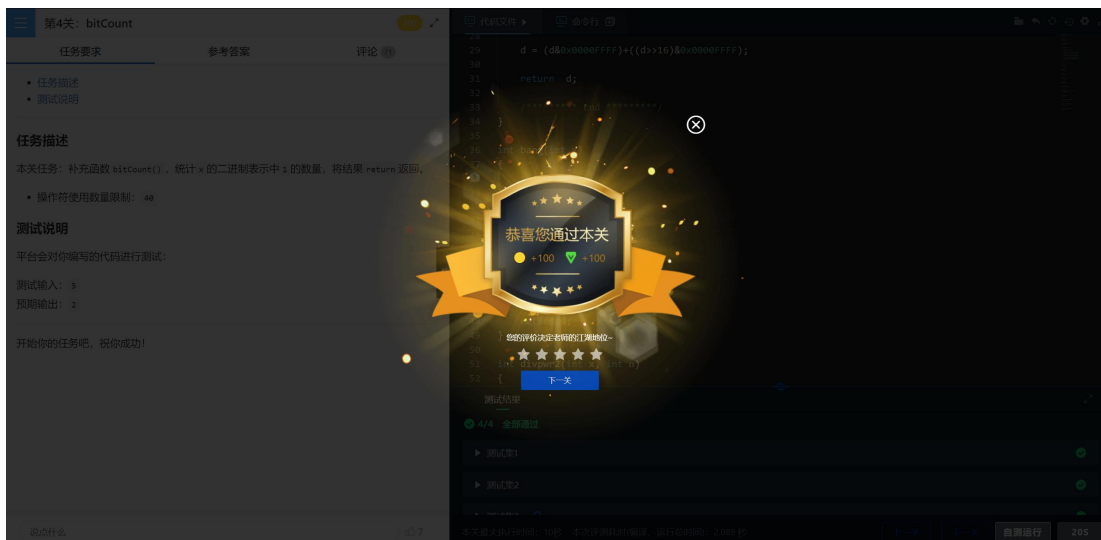
    d = (d&0x0000FFFF)+((d>>16)&0x0000FFFF);

    return d;

    /******* End *****/
}
```

2) 实验过程: 计算整数二进制表示中 1 的个数的函数 bitCount。这个函数通过一系列位操作来高效地完成计算, 其核心思路是将相邻的位分组并相加, 逐步减少每组的位数, 直到最后得到 1 的总数。

3) 实验结果:



5、Bang

1) 实验代码:

```
int bang(int x)
{
    /******* Begin *****/
    x = x | (x>>16);
    x = x | (x>>8);
    x = x | (x>>4);
    x = x | (x>>2);
    x = x | (x>>1);
    return (~x)&1;

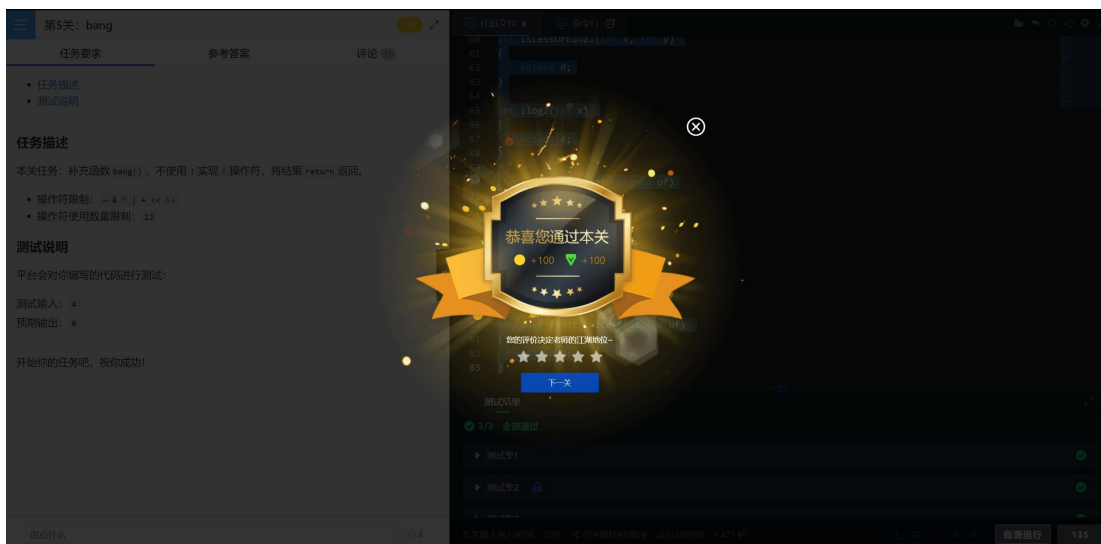
    /******* End *****/
}
```

2) 实验过程:

需要确定整数 x 是否只有一个位是 1，所以对 x 依次进行合并，在经过合并后，如果 x 只有一个位是 1，那么 x 将是一个非零值。取反操作 $\sim x$ 将把这个非零值变为 0，因为只有一个位是 1，其他位都是 0。然后，与 1 进行“与”操作 $(\sim x) \& 1$ 将得到 0，因为 0 与任何数进行“与”操作都是 0。

如果 x 不是 2 的幂，那么在合并过程中会有多于一个的位变成 1，取反操作 $\sim x$ 将不会把所有位都变为 0，至少有一个位是 1。与 1 进行“与”操作 $(\sim x) \& 1$ 将得到 1，因为 1 与 1 进行“与”操作是 1。

3) 实验结果:



6、tmin

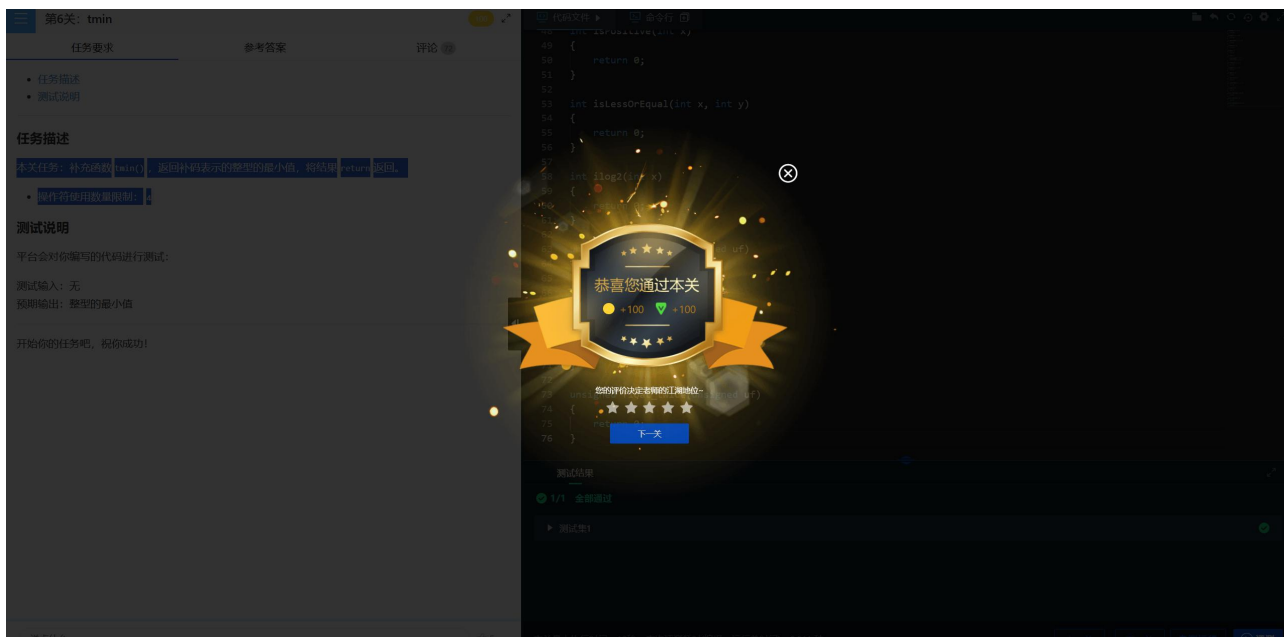
1) 实验代码:

```
int tmin(void)
{
    /****** Begin *****/
    return 1<<31;
    /****** End *****/
}
```

2) 实验过程:

进行位移操作，左移 31 位。

3) 实验结果:



7、fitsBits

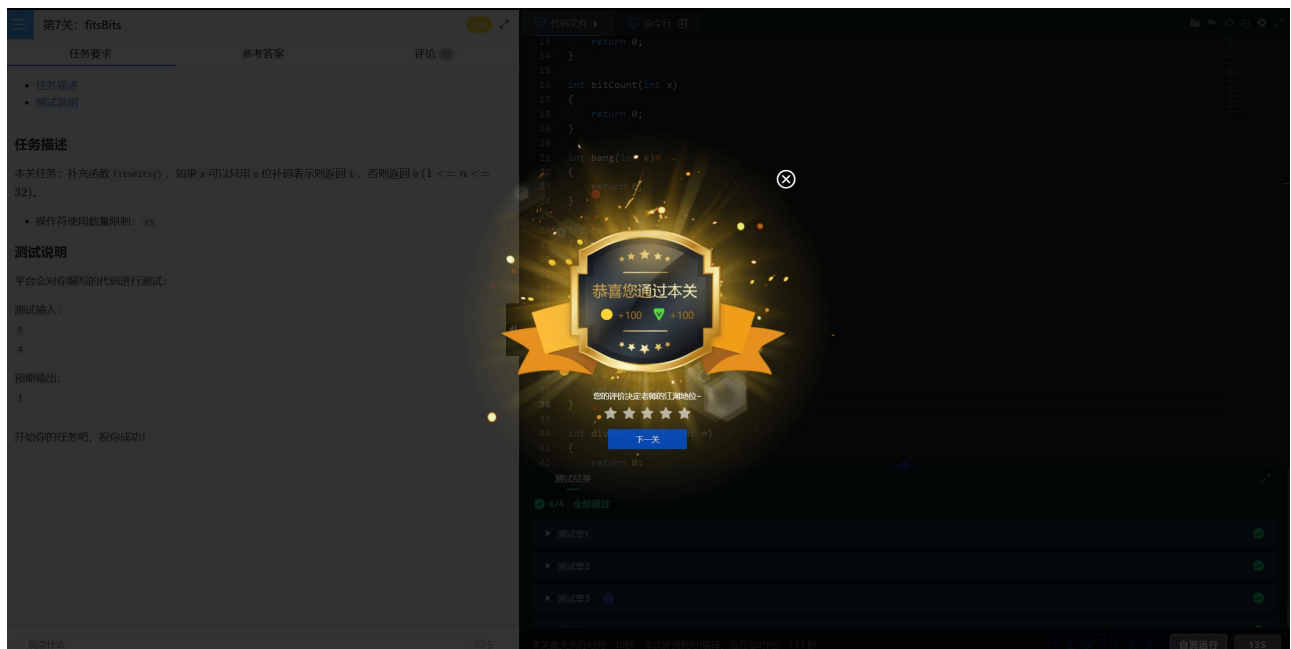
1) 实验代码:

```
int fitsBits(int x, int n)
{
    int s = 32-n;
    int m = x;
    m = m<<s;
    m = m>>s;
    return !(m^x);
}
```

2) 实验过程:

检查一个整数 x 是否能在 n 位二进制数中完整表示, 先计算位移, 左移再右移, 使得 x 的最高有效位移动到 32 位整数的最高位。再将左移后的数右移 s 位, 如果 x 能够被 n 位完整表示, 那么这个操作会恢复 x 的原始值。最后进行检查。

3) 实验结果:



8、divpwr2

1) 实验代码:

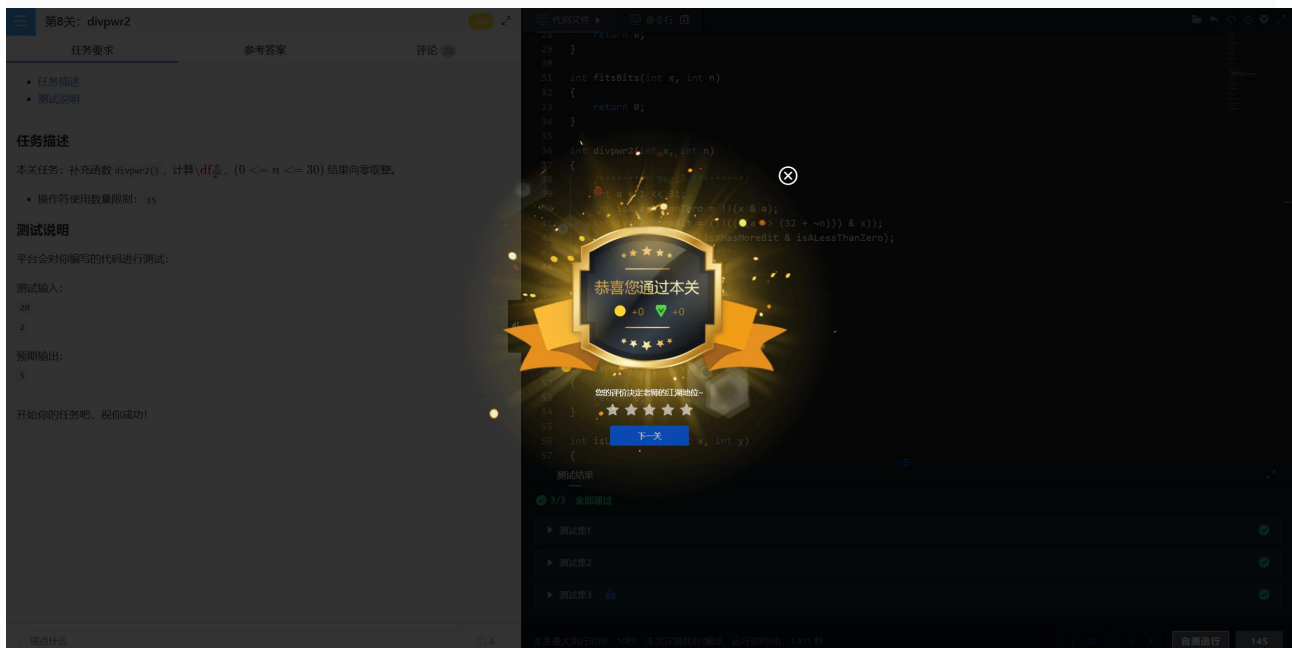
```
int divpwr2(int x, int n)
{
    /******* Begin *****/
    int a = 1 << 31;
    int isALessThanZero = !(x & a);
    int isXHasMoreBit = (!(~(a >> (32 + ~n))) & x));
    return (x >> n) + (isXHasMoreBit & isALessThanZero);
    /******* End *****/
}
```

2) 实验过程:

当 $x > 0$, 需向右移动 n 位;

而当 $x < 0$, 需要将 x 加上偏置 $(x + 1 < n) + \sim 0$;

3) 实验结果:



10、isPositive

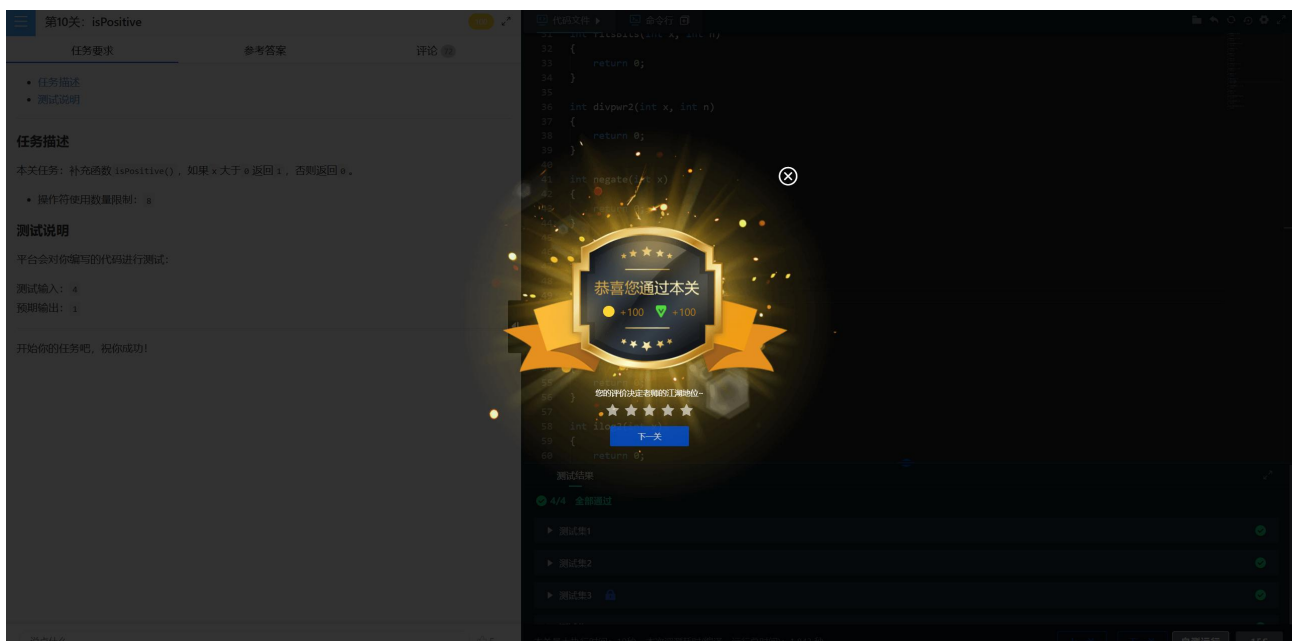
1) 实验代码:

```
int isPositive(int x)
{
    int s = x >> 31;
    int r = !(x < 0);
    return !s & r;
}
```

2) 实验过程:

大于 0 的数, 是非 0 正数, 因此需要根据符号位确定是正数, 还要不是 0。

3) 实验结果:



11、isLessOrEqual

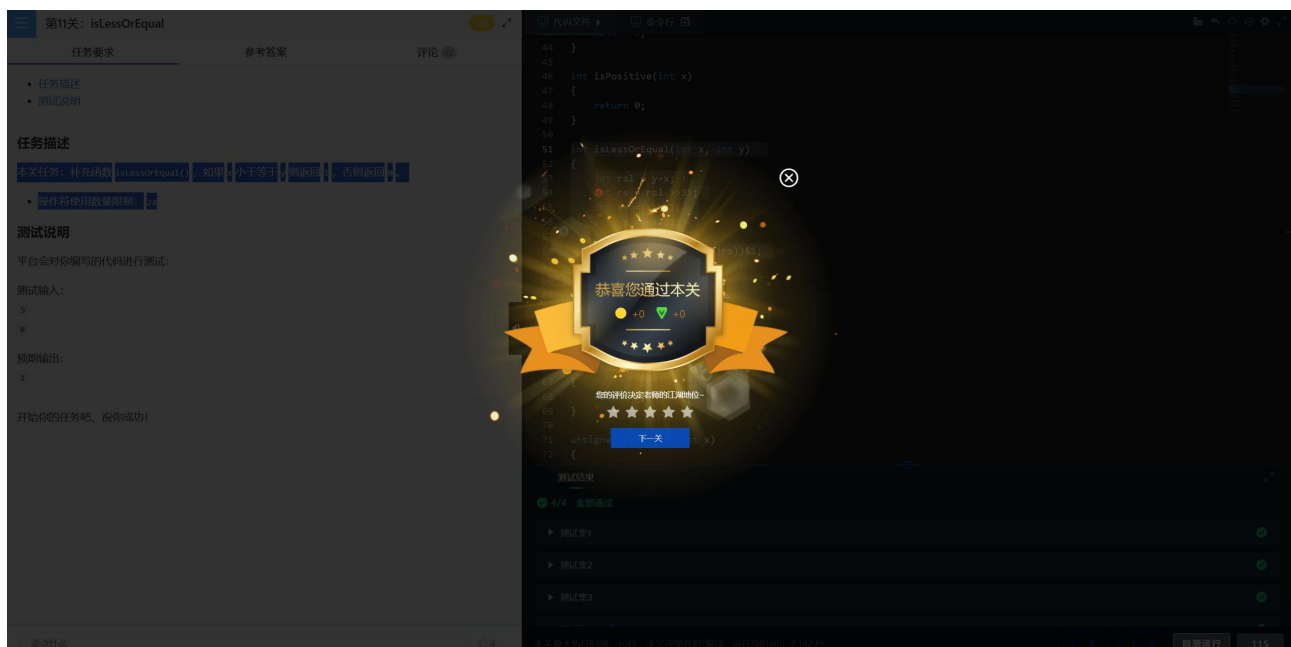
1) 实验代码:

```
int isLessOrEqual(int x, int y)
{
    int rs1 = y-x;
    int rs = rs1 >>31;
    int xs = x>>31;
    int ys = y>>31;
    int ds = xs^ys;
    return ((ds&xs)|(!ds&!rs))&1;
}
```

2) 实验过程:

转化成两个问题, 如果 x, y 符号相同, y-x 的值是正数, 如果 x, y 符号不同, 那么 x 是负数。

3) 实验结果:



12、ilog2

1) 实验代码:

```
int ilog2(int x)
{
    int c = 0;
    c = (!! (x >> 16)) << 4;
    c = c + (!! (x >> (8+c))) << 3;
    c = c + (!! (x >> (4+c))) << 2;
    c = c + (!! (x >> (2+c))) << 1;
    c = c + (!! (x >> (1+c)));

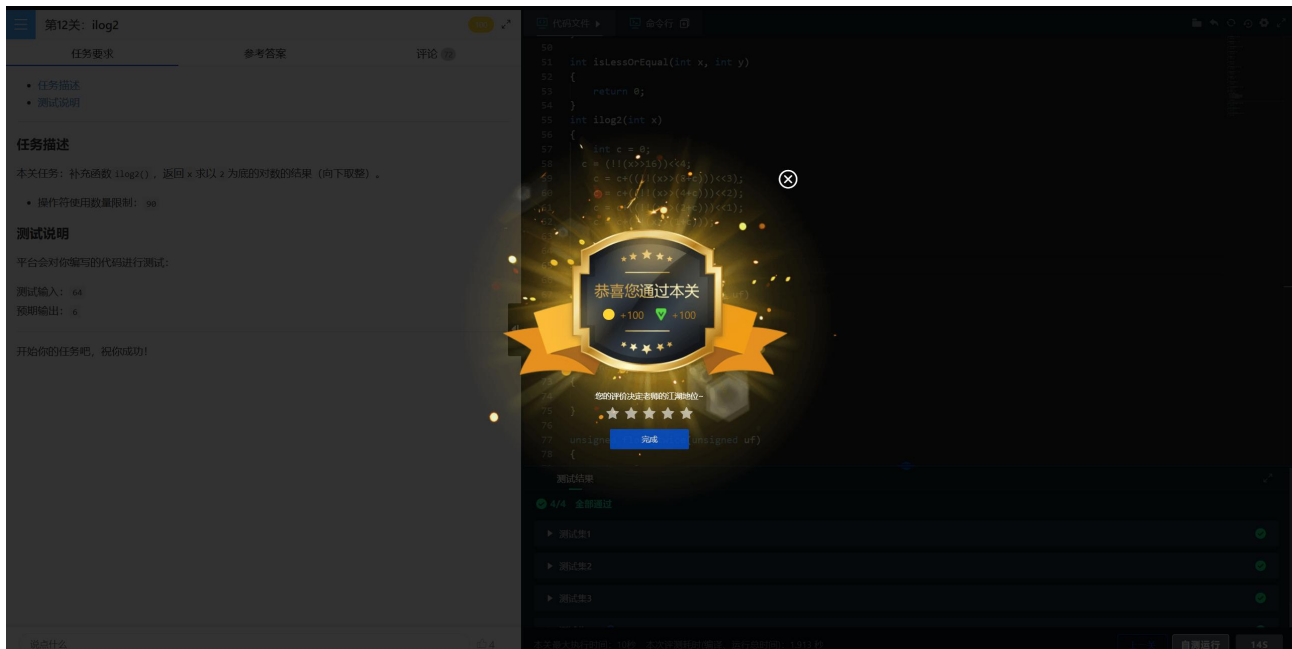
    return c;
}
```

```
}
```

2) 实验过程:

先看最高位 1 是不是在高 16 位, 可以利用 $!!(x \gg 16)$ 表达式来计算, 如果 $!!(x \gg 16)$ 是 1, 表示在高 16 位, 因此暂时将结果设置成 $c=16$; 之后再计算 $!!(x \gg (8+c))$ 的值, 如果结果仍然为 1, 那么就 将结果暂时设置成 $c=c+3$; 最后计算 $!!(x \gg (1+c))$ 的结果。

3) 实验结果:



13、float_neg

1) 实验代码:

```
unsigned float_neg(unsigned uf)
{
    /******* Begin *****/

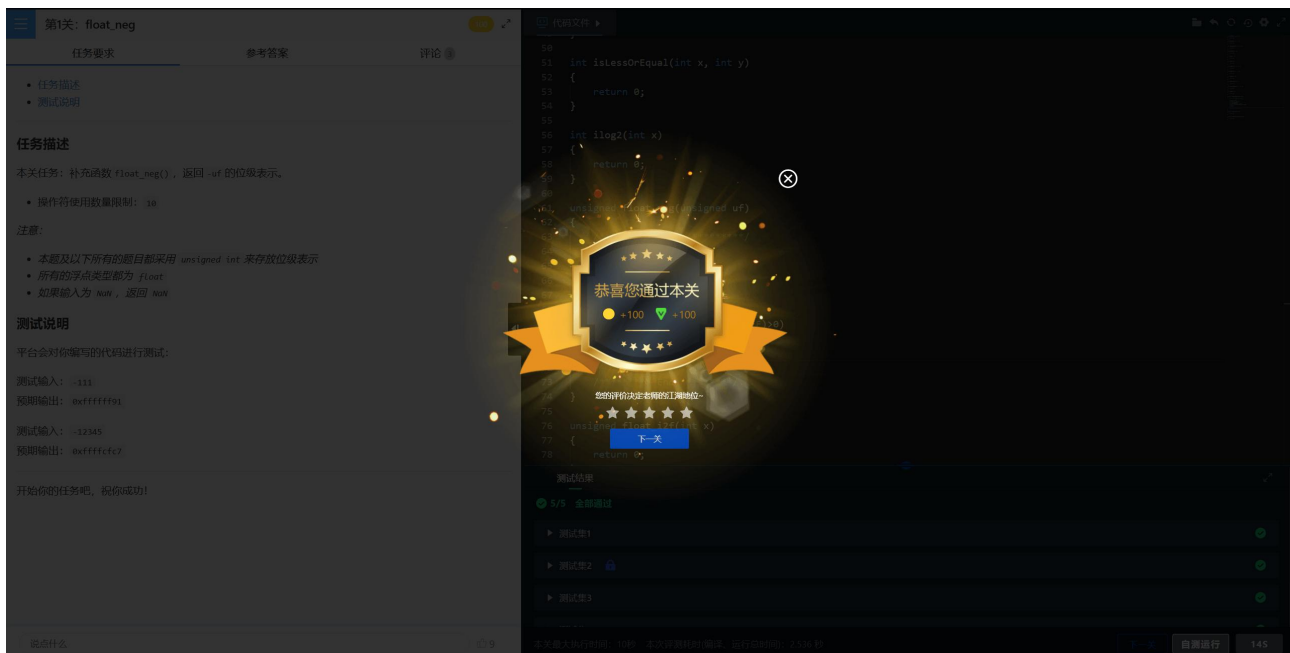
    unsigned r=uf;
    unsigned t;//符号取反
    r = r^0x80000000 ;
    t = (uf>>23)&0xff;//非数
    if (t == 0xFF && (uf&0x7FFFFFFF)>0)
        r = uf;
    return r;

    /******* End *****/
}
```

2) 实验过程:

通过按位异或操作 $\wedge$ 将 r 的最高位 (符号位) 取反。计算 t 的值, $t = (uf \gg 23) \& 0xff$ 。这一步是将 uf 右移 23 位, 然后与 $0xff$ 进行按位与操作。检查 t 是否等于 $0xFF$, 然后检查 uf 的尾数部分 ($uf \& 0x7FFFFFFF$) 是否大于 0。如果这两个条件都满足, 那么原始浮点数是 NaN。

3) 实验结果:



14、float_i2f

1) 实验代码:

```

unsigned abs = x;
unsigned s = 0x80000000 & x;
unsigned tem = 0x40000000;
unsigned exp_sign = 0;
unsigned exp = 0;
unsigned frac;
unsigned c = 0;
if (x == 0)
    return x;
else if (x == 0x80000000)
    return (s + (158 << 23));
else {
    if (x < 0)
        abs = -x;
    while (1) {
        if (tem & abs)
            break;
        tem = tem >> 1;
        exp_sign = exp_sign + 1;
    }
    frac = (tem - 1) & abs;
    if (31 - 1 - exp_sign > 23) {
        int i = 30 - exp_sign - 23;
        if ((frac << (31 - (i - 1))) == 0x80000000) {
            if ((frac & (1 << i)) != 0)

```

```

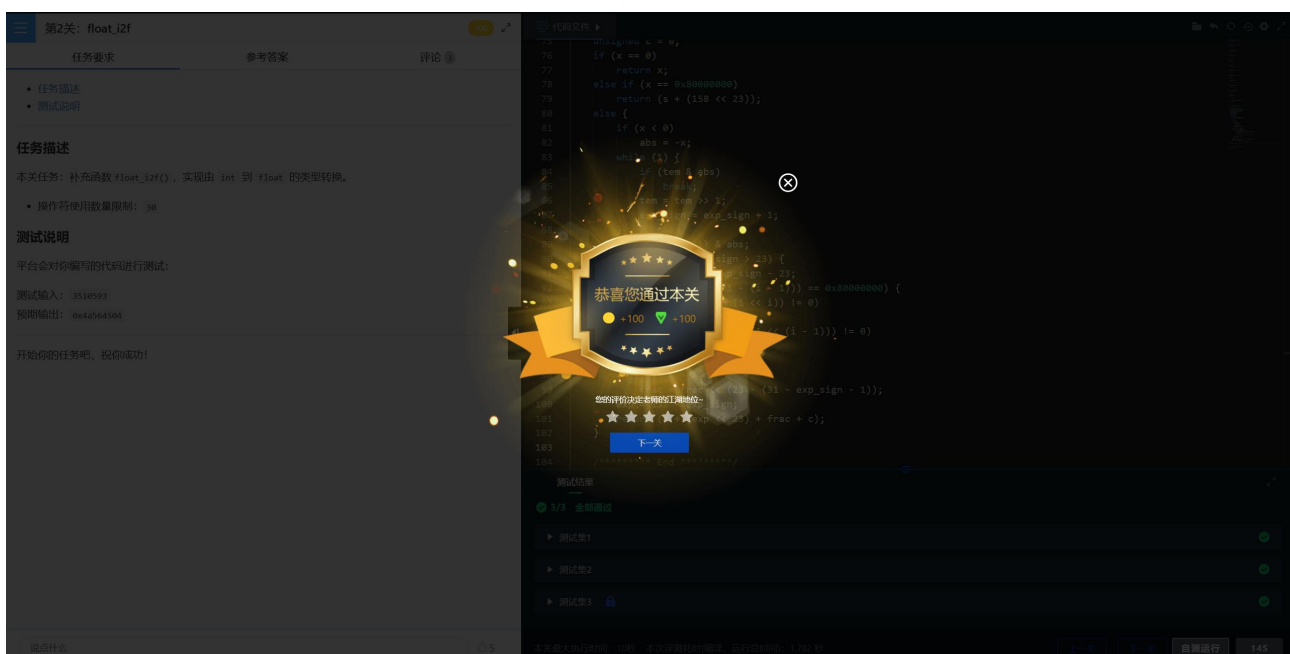
        c = 1;
    } else if ((frac & (1 << (i - 1))) != 0)
        c = 1;
    frac = frac >> i;
} else
    frac = frac << (23 - (31 - exp_sign - 1));
exp = 157 - exp_sign;
return (s + (exp << 23) + frac + c);
}

```

2) 实验过程:

这段代码是将一个 32 位有符号整数转换为对应的单精度浮点数表示。主要有以下几个步骤：确定符号位、找到最高位 1、计算尾数、调整尾数和指数、处理进位、组合符号位、指数和尾数、返回结果。

3) 实验结果:



15、float_twice

1) 实验代码:

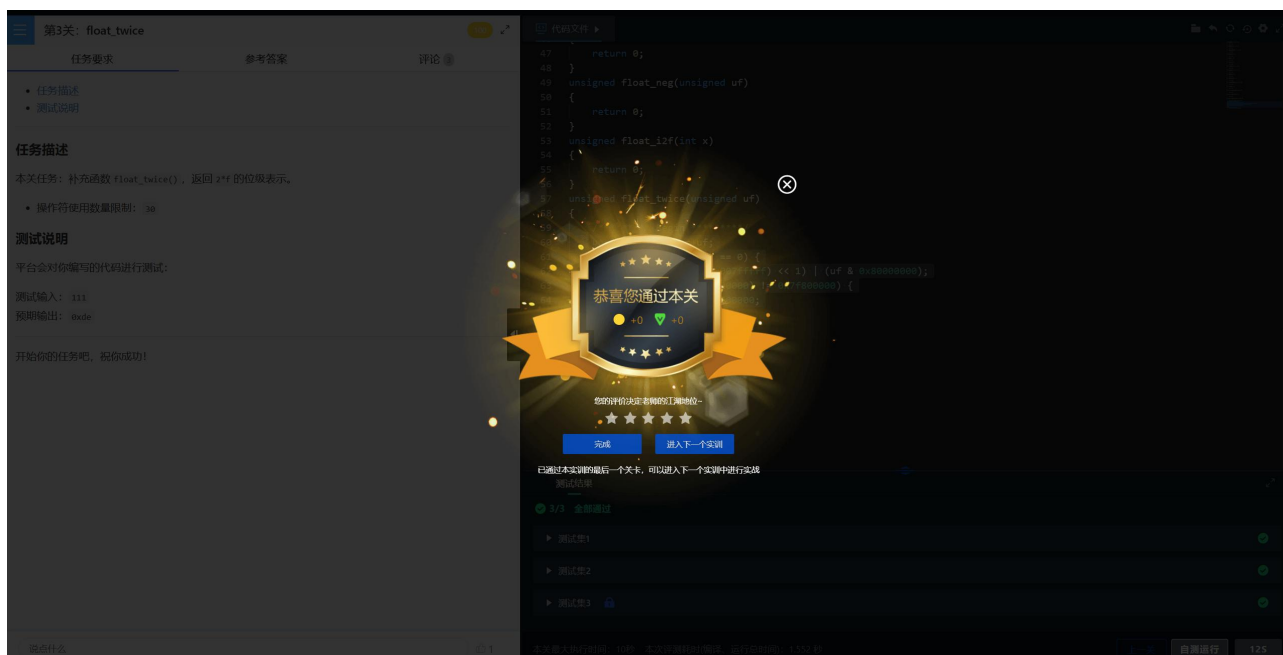
```

unsigned result = uf;
if ((uf & 0x7f800000) == 0) {
    result = ((uf & 0x007fffff) << 1) | (uf & 0x80000000);
} else if ((uf & 0x7f800000) != 0x7f800000) {
    result = uf + 0x00800000;
}
return result;

```

2) 实验过程: 这段代码是将一个表示浮点数的无符号整数 uf 转换为另一种形式，将输入的无符号整数 uf 赋值给 result。主要步骤为处理非规格化数、处理特殊值、返回结果。

3) 实验结果:



五、实验总结

通过这次实验，我加深了对位运算的理解，提高了运用二分查找算法的能力，并且巩固了对浮点数表示法的记忆，包括各种二进制编码形式，如原码、补码和反码等。在这个过程中，我还复习了 C 语言的相关知识，这有助于提升我的逻辑思考能力。特别是最后两个实验，它们让我认识到编程时需要全面考虑各种因素，细致周到地思考问题，并且要有严谨的逻辑思维。此外，通过测试来优化和完善代码也是一种有效的实践方法。